Skeletonizing the worm

Supervisor: Professor Sam Braunstein

Author: Harry Yeh

Date: 16th May 2024

BEng Computer Science

Department of Computer Science

University of York

Table of Contents

1 Executive Summary

1.1 Project Aim and Methods

A research study was conducted at the University of Leeds to compare the virtual locomotion of simulated worms with real worms moving on agar plates under varying conditions, with the objective to validate existing models and formulate new hypotheses. The aim of my project is to develop a supplementary skeletonization programme to assist in the research. The programme is designed to generate the skeleton as a digital representation of a microscopic worm that accurately describes the posture and motion of the worms from the provided video examples. The main method of this project involves four main phases: Preparation, Skeletonization, Adjustment, and Composition. The image is initially pre-processed to undergo skeletonization. Necessary adjustments are then made to the skeleton mask before it is composited onto the original image. The result can be shown in the worm video with its green digital backbone being overlaid. Refer to Figure 5.4 on page 22 for the example.

1.2 Motivations

Curiosity and exploration: Pursuing this kind of project can be a way to satisfy my curiosity and explore new technologies and techniques. This project has given me a gentle introduction and a basic understanding of Computer Vision. It offers the opportunity for a challenging and rewarding experience, allowing me to learn how to analyse and process images and videos, and to develop new algorithms and applications. It provides a sense of accomplishment that fuels my passion for exploration and innovation!

Problem-solving: The knowledge gained from this project can help to address pressing real-world issues, including but not limited to medical diagnosis, quality control, and surveillance. By pursuing this project, I can leverage my problem-solving skills and knowledge to make meaningful contributions to the needs of the society and the nation, advancing progress and innovation in critical areas.

Career development: This project presents an opportunity for me to gain valuable expertise that is highly sought after in various industries such as healthcare, manufacturing, and technology. By pursuing this project, I can

enhance my skills and knowledge, making me a more competitive candidate in the job market. Additionally, it allows me to build a strong portfolio and showcase my capabilities to potential employers, opening doors to exciting career opportunities.

1.3 Issues

Ethical and Legal: The techniques are widely used in medical imaging to analyse and diagnose diseases. For example, skeletonization can be used to extract the skeletal structure of bones and organs, which can help doctors identify abnormalities and make more accurate diagnoses and treat the patient accordingly. However, this may give rise to ethical issues related to privacy [1] and autonomy if informed consent and transparency are concerned. Additionally, it could raise liability issues if it fails to perform as expected, especially when patient safety is involved.

Social: The technique can be used in surveillance and security systems to detect and track objects, which can help prevent crime and improve public safety. For example, skeletonization can be used to extract the features of people and vehicles, which can help identify suspicious movement, behaviour and trigger alerts [1]. However, it could perpetuate existing biases and stereotypes if the algorithms are not designed to be fair and unbiased [1]. This can have social implications related discrimination and fairness.

Commercial and Legal: The technique can be used in manufacturing and quality control to inspect products and ensure that they meet quality standards. For example, skeletonization can be used to extract the features of complex objects and compare them to a reference model, which can help identify defects and improve product quality. However, if it is used to create derivative works based on copyrighted material, it could raise commercial and legal issues related to innovation and intellectual property. Companies with closely similar products or technologies may find themselves competing against each other and must protect their patents or trademarks to avoid copyright infringement and unauthorised replication.

Professional: Ultimately, technologies inspired by Computer Vision can raise professional issues related to competency and standards. Users are responsible to ensure that they are competent and qualified to use the technology. The technology should be used in a way that is safe, effective, and respectful of people's rights and privacy.

2 Main body

2.1 Introduction

2.1.1 Project Aim

A research study was conducted at the University of Leeds to compare the virtual locomotion of simulated worms with real worms moving on agar plates under varying conditions, with the objective to validate existing models and formulate new hypotheses. The aim of the project is to develop a supplementary skeletonization programme to assist in the research. The programme is designed to generate the skeleton as a digital representation of a microscopic worm that accurately describes the posture and motion of the worms from the provided video examples. This information can be used for further analysis.

2.1.2 Caenorhabditis Elegans

The above-mentioned worm is Caenorhabditis elegans (“C. elegans”) which is a species of roundworm called “nematode”. They are about 1 mm long; they are not parasites and are free-living. They live in soil and feed on bacteria. C. elegans is a model organism, used to study animal development and behaviour. It is the first multicellular organism for which scientists have been able to sequence its whole genome. [2]

Why is C. elegans the model organism? “The organism’s neuron circuit, made up of only 302 neurons, is extremely primitive especially when compared to the approximately 100 billion neurons and the 60 trillion connections between them, known as synapses, that make up the average human brain. However, as a result of its simplicity, transparent nature and the well- characterized cell lineages, the entire pattern of neuron connections or the connectome of the C. elegans has been mapped.” [3, 0:47 – 1:18]

“Despite the relatively simple neuronal system present within C. elegans, it displays numerous complex and primitive behaviours. Primitive behaviours include feeding, locomotion and reproduction. Complex behaviours include learning, mating or social behaviours. Researcher studies behaviour of these

worms by observing how they are attracted to food and chemicals. Ultimately, behaviours reflect the activity of the nervous system. Despite their small size and simplicity, C. elegans are very useful to science both understand basic biological mechanisms and also to study the basic mechanisms of disease, that translate understanding how to fight disease in humans." [3, 1:19 – 2:02]

The use of this model organism for experiment and research has contributed to significant achievement in the field of neuroscience.

2.1.3 Applications of Skeletonization

Skeletonization serves as a powerful tool for feature extraction, shape analysis, and pattern recognition. By converting complex images into simplified skeletal representations, researchers can effectively capture the underlying structure and topology of objects, regardless of their size, orientation, or scale [4]. This technique holds significant importance and is widely employed in various fields, including Biology and Computer Vision. By pursuing this project, I can leverage my problem-solving skills and knowledge to make meaningful contributions to the needs of the society and the nation, advancing progress and innovation in critical areas. The knowledge gained from this project can help to encourage new discoveries, such as research, and address pressing real-world issues, including but not limited to medical diagnosis and surveillance.

In Biology, skeletonization plays a crucial role in understanding the underlying structures and functions of organisms. In the field of anatomy, the skeletal system provides the framework that supports and protects the body's soft tissues and organs [5]. By studying skeletal structures, researchers gain insights into evolutionary relationships, locomotion patterns, and physiological adaptations across species. In the field of botany, skeletonization helps explain the intricate vascular networks of plants, revealing the pathways for nutrient transport and resource allocation [6]. By skeletonizing plant leaves or vascular tissues, researchers can investigate vein patterns, identify adaptive strategies to environmental conditions, and study the mechanisms underlying plant growth and development.

In Computer Vision, skeletonization is used in applications, including medical imaging, robotics, fingerprint recognition, and document analysis. In medical

imaging, skeletonization can help with the detection and analysis of anatomical structures, such as blood vessels, nerves, and bones, assisting in medical diagnosis and treatment, including X-ray and MRI. Similarly, in robotics, skeletonization is used to help the robots to perceive and navigate complex environments by extracting meaningful features from their sensor data, enabling autonomous exploration and tasks [7]. In surveillance and forensics, skeletonization can facilitate the recognition of various activities or gestures performed by individuals in a surveillance video [1]. By focusing on skeletal movements, algorithms can classify actions such as walking, running, loitering, or even gestures indicating potential threats. Additionally, skeletonization reduces the amount of data needed to represent objects in a scene, making it more efficient to process and analyse surveillance footage in real-time. This can lead to faster response times and more effective monitoring of security threats.

As technology continues to evolve, the applications of skeletonization are likely to expand further, driving new insights into the sophisticated structures of the world around us.

2.1.4 Motivations

There are also personal motivations to pursue this project other than the problem-solving motivations mentioned above. Pursuing this kind of project can be a way to satisfy my curiosity and explore new technologies and techniques. This project will give a gentle introduction and a basic understanding of Computer Vision. It offers the opportunity for a challenging and rewarding experience, allowing me to learn how to analyse and process images and videos, and to develop new algorithms and applications. As a bonus, it will provide a sense of accomplishment that fuels my passion for exploration and innovation!

This project presents an opportunity for one to gain valuable expertise that is highly sought after in various industries such as healthcare, manufacturing, and technology. By pursuing this project, one can enhance my skills and knowledge, making them a more competitive candidate in the job market. Additionally, it allows one to build a strong portfolio and showcase their capabilities to potential employers, opening doors to exciting career opportunities.

2.1.5 Thinning Algorithms

One of the most widely used techniques used in image skeletonization is the thinning algorithm. It often requires parameter tuning, and the choice of parameters can affect the quality of the output. It may introduce biases or inaccuracies, especially if the algorithm is not robust or if the input data is incomplete or biased. Simplified representations obtained through skeletonization can sometimes be misinterpreted, especially if users are not fully aware of the limitations of the process or if they overlook important factors that were omitted during simplification. This can lead to misleading results or perpetuation of existing biases in decision-making processes. Selecting appropriate parameters may require domain expertise and experimentation.

Therefore, it is important to study and explore into various thinning algorithms which have their own approaches and computational characteristics. Variations of thinning algorithms are further discussed under Literature Review.

2.2 Literature Review

2.2.1 What is a thinning algorithm?

Thinning algorithms are commonly used in applications especially where extracting essential features from complex shapes is necessary for further analysis or classification. A thinning algorithm is a technique used in image processing to reduce the thickness of objects within an image by iteratively removing pixels from their boundaries based on a set of rules or conditions until they are reduced to single-pixel-wide skeleton representation. The representation preserves the geometric characteristics and spatial relationships of the original object, which can be used for shape matching, object recognition, and image interpretation.

2.2.2 Sequential or Parallel?

Thinning algorithms can be implemented sequentially or in parallel. Sequential algorithms process pixels in a particular order or sequence. In contrast, parallel algorithms divide the image into smaller regions and process them concurrently, potentially improving efficiency and achieving significant speedups. However, communication between processes may be necessary to maintain connectivity and coherence. Designing efficient parallel thinning algorithms requires careful consideration of data dependencies, load balancing, and synchronization to ensure correct results and optimal performance. This means they may require more memory and computational resources compared to sequential algorithms.

2.2.3 Zhang-Suen Thinning Algorithm

Zhang-Suen algorithm is a popular parallel thinning algorithm we will explore and take inspiration from the Hit-and-Miss transform method on which it is built. Hit-and-Miss transform is a mathematical morphology technique used to detect specific patterns or shapes within binary images. This technique involves the use of structuring elements to match certain patterns within the image. Zhang-Suen algorithm consists of two main steps, each using a structuring element. These elements are used to convolve with every pixel of the underlying image simultaneously. Each element is a 3x3 matrix with

central anchor P1 surrounded by 8 neighbours P2 to P9, as shown in Figure 2.1.

P9	P2	P3
P8	P1	P4
P7	P6	P5

Figure 2.1: *outline of the structure element*

In the first step, the algorithm identifies and removes pixels at the south-east boundaries and north-west vertex of the objects. A pixel is set to white if conditions of the structuring element A through E listed below are met. These conditions are crucial for maintaining the connectivity and structure of the image while thinning. In the second step, the algorithm performs the same process but from the opposite direction with conditions that are same as the first step, except that conditions D (east) and E (south) are now replaced with F (north) and G (west) [8].

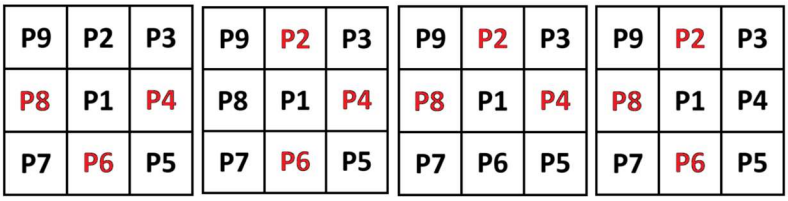

Figure 2.2: *south – east – north – west from left to right*

Conditions [9, Steps 1 and 2]:

A. current pixel is black and has eight neighbours

B. number of black neighbours is within the range of 2 to 6

C. number of transitions from white to black in the sequence P2,P3,P4,P5,P6,P7,P8,P9,P2 is 1

D. at least one of P2 and P4 and P6 is white

E. at least one of P4 and P6 and P8 is white

F. at least one of P2 and P4 and P8 is white

G. at least one of P2 and P6 and P8 is white

This iterative process continues until no further changes occur in either step, indicating that the thinning process has reached a stable state. At this point, the algorithm terminates, and the resulting image represents a skeletonized version of the image.

2.2.4 Comparison of Algorithms

Guo-Hall, Hilditch's, Holt's are examples of the many other thinning algorithms that have been developed as improved versions or alternatives to the Zhang-Suen thinning algorithm [10]. While these different thinning algorithms are effective in reducing shapes to their skeletons, choosing between these algorithms depends on the specific requirements of the programme and the desired trade-offs between computational efficiency and skeleton quality. Zhang-Suen algorithm is simpler and more computationally efficient, making it suitable for real-time applications where speed is crucial. On the other hand, the improved alternatives offer better performance in terms of skeleton quality, producing skeletons with fewer artifacts and better connectivity.

Given the simplicity of the programme and limited computational resources in the system, computational efficiency will be prioritised over quality to prevent performance bottleneck. Therefore, alternatives have been ruled out and will not be discussed. Instead, I will opt directly for a sequential implementation of the Zhang-Suen algorithm. The original parallel implementation for this algorithm can be made sequential by using an alternative structuring element with equivalent effect. One possible method is to merge and compress the two elements into one.

However, in cases where quality becomes a concern, employing suitable image processing techniques can effectively enhance overall performance, mitigating the algorithm’s limitation.

2.3 Background

I implement the chosen thinning algorithm for the skeletonization programme using software development tools such as OpenCV and MATLAB. OpenCV is an open-source Computer Vision library of programming functions mainly aimed at real-time computer vision and MATLAB is a proprietary programming environment and programming language widely used in academia and industry for numerical computing and data analysis.

OpenCV offers a wide range of functionalities for image and video processing, including image manipulation, feature detection, object recognition, and machine learning integration. Its modular structure allows users to customize and extend its capabilities according to their specific requirements. Furthermore, OpenCV's modular architecture enables easy integration with other libraries and frameworks, fostering compatibility and extensibility.

MATLAB provides several toolboxes dedicated to image processing and computer vision, such as the Image Processing Toolbox and the Computer Vision Toolbox. These toolboxes offer a wide range of algorithms, functions, and pre-built applications for various tasks, from basic image enhancement to complex object recognition. MATLAB, although highly versatile with its extensive toolbox, may sometimes lack the flexibility offered by OpenCV, especially when it comes to integrating with external libraries or deploying to different platforms.

Performance is a critical factor in computer vision applications, especially when dealing with real-time processing or large-scale datasets. OpenCV, being primarily implemented in C++, has emphasis on optimization and hardware acceleration which offers high performance and efficiency, making it suitable for real-time applications. On the other hand, MATLAB, while providing optimized functions for many common computer vision tasks, may not always match the performance of OpenCV due to its lack of low-level optimization.

MATLAB's user-friendly interface and comprehensive documentation make it an attractive choice for beginners and researchers who prioritize ease of use and rapid prototyping. Its interactive IDE, along with built-in functions and visualization tools, simplifies the development process and reduces the learning curve. OpenCV, on the other hand, may have a steeper learning curve, particularly for those less familiar with C++ or Python programming

languages. However, its extensive community support and abundance of online resources has mitigated this challenge.

Last but not least, one other significant factor to consider is the cost and licensing model associated with each platform. OpenCV is free and open-source, allowing unrestricted usage, modification, and distribution for both commercial and non-commercial purposes. In contrast, MATLAB requires a commercial license for full access to its features and toolboxes, potentially causing financial challenges for individuals or organizations with limited budgets.

Since I am developing a simple and straightforward real-time programme, opting for the free and open-source OpenCV over MATLAB is preferable for the comparisons explained above. OpenCV itself is primarily implemented in C++, but has interfaces for various languages, including Python. The standard Python os module and NumPy library will be used alongside with OpenCV.

The os standard module in Python is a powerful utility for interacting with the operating system. It provides a way to access operating system functionalities such as file system operations, environment variables, process management, and more. With the module, you can perform tasks like navigating directories, creating, moving, or deleting files, executing shell commands, and retrieving information about the system environment. Additionally, it abstracts away platform-specific differences, making it easier to write cross-platform code. Whether you're building a file manipulation tool, managing processes, or simply need to interact with the underlying operating system, the module offers a comprehensive set of functions to help you accomplish your tasks efficiently and reliably in Python.

The NumPy module in Python is an essential library for numerical computing, offering a powerful array object and a variety of tools for working with these arrays. This module is widely used in scientific and engineering applications due to its ability to perform array operations with high performance. NumPy provides a wide range of mathematical functions for array manipulation, linear algebra, random number generation, and more. Its seamless integration with other scientific computing libraries makes it a crucial tool for data analysis, machine learning, and computational tasks in Python. With NumPy, developers can efficiently process large datasets, perform complex mathematical computations, and build high-performance applications with ease.

I start off the implementation by importing relevant module, libraries and video sources. A specific microscopic-worm video can be selected by using the 'os.listdir()' method on the relevant directory and specifying its index. Refer to Figures 3.1 and 3.2 for example.

```
import os
import cv2 as cv
import numpy as np
```

Figure 3.1: *import relevant module and libraries*

```
directory = "sample_movies"
index = 7
video = os.listdir(directory)[index]
cap = cv.VideoCapture(directory + "\\" + video)
```

Figure 3.2: *import worm source*

2.4 Methodology, Design, and Implementation

2.4.1 Programme Summary

The main method of this project involves four main phases: Preparation, Skeletonization, Adjustment, and Composition. Once the video is imported, it is fragmented into frames. A captured frame has dimensions of n x m x c, where n represents the height, m represents the width, and c represents the number of channels. The frame is represented by a 3-dimensional NumPy array. The first dimension contains n rows. The second dimension consists of m pixels stored in each row. Each pixel is composed of c values, which in this case are 3 values in the range of 0 to 255 depicting the intensity for the colours blue, green and red (BGR/RGB). Each frame is initially pre-processed to undergo skeletonization, generating the digital skeleton of the worm. Colour adjustment is then made to the skeleton mask before it is composited onto the original image.

```
# Collections of threshold values used
t_values = set()

while cap.isOpened():

    ret, frame = cap.read()
    if not ret:
        print("Can't receive frame (stream end?). Exiting ...")
        break

    # Preparation
    t, thresh = preprocess(frame)
    t_values.add(t) # add the threshold value to collection

    # Skeletonization
    skeleton = skeletonize(thresh)

    # Adjustment: Change the color of the skeleton for feature clarity
    skeleton_green = cv.cvtColor(skeleton, cv.COLOR_GRAY2BGR)
    black_threshold = 10
    white_pixels = np.all(skeleton_green > black_threshold, axis=-1)
    skeleton_green[white_pixels] = [0,255,0]

    # Composition of the overlay and original
    composition = frame.copy()
    composition[skeleton_green != 0] = skeleton_green[skeleton_green != 0]

    cv.imshow('Skeleton', composition)

    if cv.waitKey(1) == ord('q'):
        break

cap.release()
cv.destroyAllWindows()
```

Figure 4.1: *The four main phases in the programme*

2.4.2 Preparation

The 'preprocess' procedure in the programme is designed to prepare the image for skeletonization by applying thresholding and denoising techniques, with the aim of achieving segmentation. These techniques are commonly used as preprocessing steps to enhance images or prepare them for subsequent operations.

```
def preprocess(frame):

    # Apply threshold
    frame = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
    t_value, frame = cv.threshold(frame, 0, 255, cv.THRESH_BINARY_INV + cv.THRESH_OTSU)

    # Remove noise and enhance the subject
    kernel = np.ones((3, 3), np.uint8)
    frame = cv.morphologyEx(frame, cv.MORPH_OPEN, kernel)

    return t_value, frame
```

Figure 4.2: *procedure for Preparation phase*

2.4.3 Greyscale and Thresholding

This first part of the procedure is crucial for creating a clear distinction between foreground and background, enabling subsequent segmentation and skeletonization of the frame.

Thresholding is an essential image processing technique used to isolate areas of interest. Grey-scaling the image is necessary before applying the threshold to achieve optimal results. The image is compressed into a single channel and converted into shades of grey, with pixel intensities ranging from 0 (black) to 255 (white), in order to depict contrast and texture. The conversion is achieved by applying the luminance coefficients (0.114, 0.587, 0.299) to the BGR values of each pixel in the image respectively.

Inverted threshold is then applied to the image, converting it into a binary image. Pixels with values higher than the threshold computed by the Otsu algorithm are set to 0, while pixels with values below or equal to the threshold are set to the maximum value, 255, from the second argument of the method.

2.4.4 Otsu's Algorithm

The intensity values of all greyscale pixels form a bimodal histogram. Two classes, background (bg) and foreground (fg) are separated by a threshold (t). Weight (w) for each class can be calculated by dividing its number of pixels by the total number of pixels in the image. Variance (σ^2) for each class can be calculated by dividing its sum of squares by its number of pixels. Intra-class variance at t can be calculated by substituting the respective weight and variance for each class in the formula shown in Figure 4.3. Threshold with the smallest intra-class is chosen as the optimal threshold value. [11], [12]

$$\sigma^2(t) = \omega_{bg}(t)\sigma^2_{bg}(t) + \omega_{fg}(t)\sigma^2_{fg}(t) \quad \text{[12]}$$

Figure 4.3: *intra-class variance for Otsu's algorithm*

2.4.5 Denoising with Morphological Operations

In Computer Vision, the presence of unwanted noise significantly impacts image quality. Noise obscures critical details and introduces false artifacts within the image, which can substantially affect how they are processed by the programme. Additionally, noise may introduce false patterns or distortions that mislead recognition and classification algorithms. By reducing noise, these algorithms can make more accurate and reliable decisions, ultimately enhancing performance in tasks like object detection or image classification. Moreover, for applications involving human interpretation, such as this project, denoising can enhance the visual appeal and clarity of images. This is particularly crucial in fields like medical imaging, where accurate visualization of details is critical for diagnosis and treatment.

Erosion and Dilation are morphological operations that can be used to reduce the noise in the microscopic frames, substantially reducing static and unnecessary shadows, providing clearer images that are crucial for edge and object detection of the worm we are seeking to identify and explore further. Erosion decreases the size of the objects in the image, whereas Dilation has the opposite effect.

These operations are extensively used in the programme, as they play a crucial role in implementing the thinning algorithm based on the principles of Hit-and-Miss transform method.

2.4.6 Hit-and-Miss Transform

During Dilation, the current pixel in the image is marked by the anchor (centre) of the element if any neighbouring pixels around the origin match, which is referred to as a “hit”. The value of the current pixel in the image is updated to the maximum value found within its neighbourhood defined by the structuring element, and effectively expands the brighter (white) regions in the image. In contrast, for erosion, it is marked only if the entire element pattern matches, which is referred to as a "fit". The value is updated to the minimum value instead, and effectively shrinks the darker (black) regions in the image. If the pattern does not match, it will result in a "miss". [13]

The mathematical formulas for dilation and erosion operations from OpenCV can be seen in Figures 4.4 and 4.5.

$$\texttt{dst}(x,y) = \max_{(x',y'):\, \texttt{element}(x',y') \neq 0} \texttt{src}(x+x', y+y')$$ [14]

Figure 4.4: *dilate() operation from OpenCV*

$$\texttt{dst}(x,y) = \min_{(x',y'):\, \texttt{element}(x',y') \neq 0} \texttt{src}(x+x', y+y')$$ [14]

Figure 4.5: *erode() operation from OpenCV*

2.4.7 Structuring Element

The effectiveness of this method relies on the design and choice of appropriate structuring elements, in the form of binary kernel, tailored to the patterns of interest in the image. As part of Preparation phase, a 3x3 structuring element filled entirely with 1s is used for the erosion operation, followed by dilation to restore the overall structure of the image. The sequence of these two operations, using ‘cv.morphologyEx()’ method with the flag ‘cv.MORPH_OPEN’, effectively reduces noise and smooth out the contour of the worm, preparing the frame for further processing. In Skeletonization phase, a 3x3 kernel in the shape of a cross "+", filled with 1s, serves as a replacement in the thinning algorithm for the two structuring elements employed in the Zhang-Suen parallel algorithm. This structuring element can be created using ‘cv.getStructuringElement()’ method with the flag 'cv.MORPH_CROSS'.

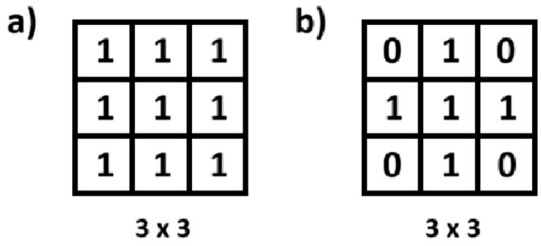

Figure 4.6a: *structuring element used in Preparation*

Figure 4.6b: *structuring element used in Skeletonization*

2.4.8 Thinning Algorithm for Skeletonization

The first step of the thinning algorithm involves subtracting the opening (erosion + dilation) of the image from the original image. This resulting image is then combined with the previous skeleton mask, using bitwise-or operation, to produce the updated skeleton mask. The original image is replaced with the current eroded image for the next iteration. Using the idea from Zhang-Suen, this algorithm is performed iteratively until no further changes can be made to the eroded image. If the image consists of only zeros, indicated by 'zeros == size', the algorithm terminates by breaking the loop with 'cond=False'. Refer to Figure 4.7.

2.4.9 Adjustment and Composition

Once skeletonization has completed, the binary skeleton mask is being converted back to BGR format, with the subject represented in [255,255,255] (white) and the background in [0,0,0] (black). Any pixel with an intensity more than the specified threshold will be considered white and assigned a new BGR colour represented by [0, 255, 0]. The threshold value is inclusive of and can be any number between 1 and 255, as all pixels are either black or white. This effectively changed the colour of the subject from "white" to this specific colour. Ultimately, these relevant pixels for the skeleton subject are selected and extracted from the mask and composited into the original frame, forming the complete composition that is subsequently displayed. Refer to Figure 4.1.

```python
# Using Hit & Miss method for skeletonization

def skeletonize(frame):

    skeleton = np.zeros(frame.shape, np.uint8)

    # Using a 3x3 "+" structure kernal
    kernel = cv.getStructuringElement(cv.MORPH_CROSS, (3, 3))

    size = np.size(frame)
    cond = True

    while cond:
        eroded = cv.erode(frame, kernel)
        temp = cv.dilate(eroded, kernel)
        temp = cv.subtract(frame, temp)

        # Image masking
        skeleton = cv.bitwise_or(skeleton, temp)

        frame = eroded.copy()
        zeros = size - cv.countNonZero(frame)
        if zeros == size:
            cond = False

    return skeleton
```

Figure 4.7: *thinning algorithm for skeletonization*

2.5 Results and Evaluation

2.5.1 Demonstration Results

A random frame of Video 8, from the source directory is used as an example to clearly demonstrate each of the phases through the programme leading to the final image.

Figure 5.1: *original image of the frame*

Due to the nature of the erosion and dilation OpenCV operations, shown in Figures 4.4 and 4.5 above, that are used in the following phase, inverted threshold is used instead of the standard threshold in Preparation to invert the white and black pixels, where white pixels represent the foreground and black pixels representing the background. When standard threshold was used, the programme failed to capture and generate the structure of the worm during Skeletonization as shown in Figure 5.2b below.

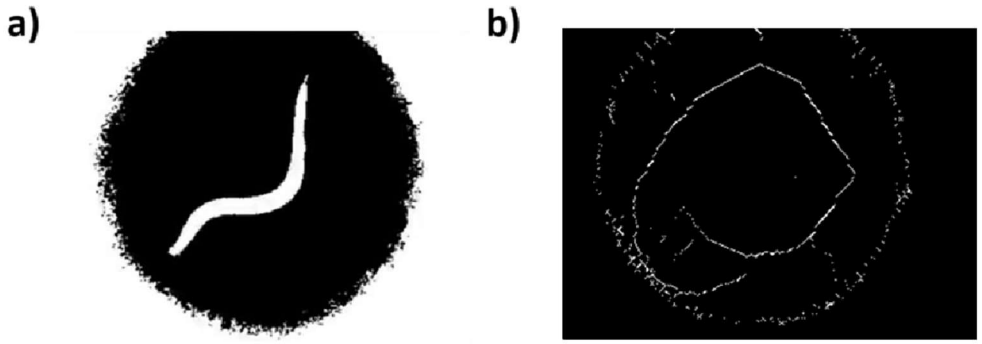

Figure 5.2a: *inverted threshold of the frame*

Figure 5.2b: *failed skeletonized outcome using standard threshold*

Successfully generated skeleton of the worm is depicted in Figure 5.3a, where it appears one-line thick and white against a black background. Colour alteration in Adjustment can be seen in Figure 5.3b with the same skeleton now in green against the same background.

a) b)

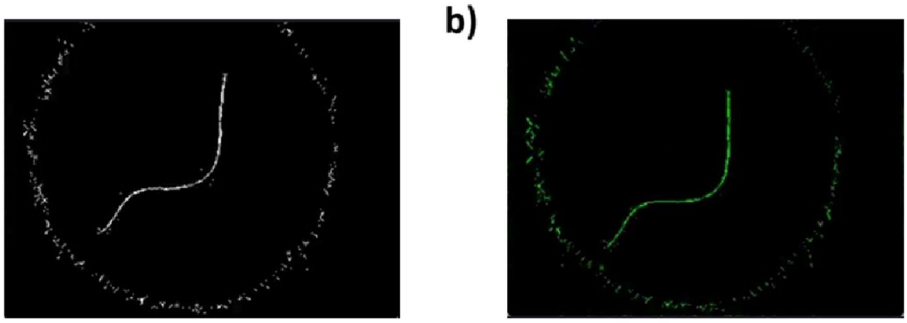

Figure 5.3a: *successful result after Skeletonization*

Figure 5.3b: *result after Adjustment*

The final result shows the composition of the worm with its digital representation.

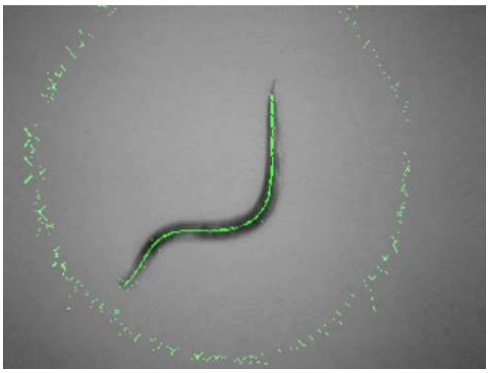

Figure 5.4: *final image of the frame*

2.5.2 Denoising Effect

Figures 5.5a and 5.5b below show a side-by-side comparison of the results with and without the denoising effect. Theoretically, the overall image quality and details of the digital backbone of the worm would be lost after denoising in Preparation. However, it is evident that the details of the digital backbone remains the same while the static around the worm in the final image has been reduced.

Although there are no significant changes seen in the results from the provided microscopic video sources, it is still a good convention to denoise an image to mitigate other unpredictable factors that may vary across different sources. This significantly improves the programme's accuracy, robustness and versatility.

a) b)

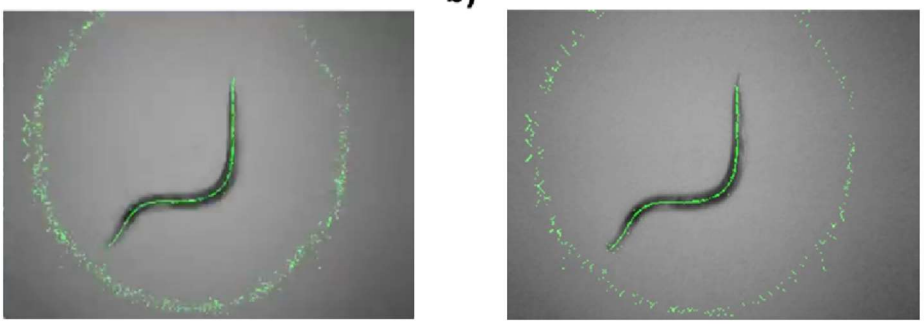

Figure 5.5a: *Final image before denoising*

Figure 5.5b: *Final image after denoising*

2.5.2 Parameter Tuning

Most parameters for the operations used in the programme are determined through trial-and-error. Trial-and-error allows one to systematically explore potential solutions and hypotheses until they find one that works. It is crucial for learning and testing, especially in situations where there's no clear solution or established method for image processing. By trying different configurations and observing their outcomes, one can gain insights into the factors influencing the problem, thus refining their understanding and skills to identify the most efficient or effective solutions to the given problem.

In Preparation, threshold value for each frame of the video is collected, shown in Figure 4.1 above, and printed out at the completion of the programme, as shown in Figure 5.6 below. Since the optimal threshold is different for each of the frames, using the Otsu's algorithm is more suitable for determining the threshold value without requiring prior knowledge of the image content, as opposed to using a fixed value for all frames. For example, Video 8 has used threshold values ranging from 114-133.

```
print(f"Set of threshold values used in Video {index+1}: {t_values}")
```

```
Set of threshold values used in Video 8: {129.0, 130.0, 131.0, 132.0, 133.0, 114.0}
```

Figure 5.6: *threshold values used for the video*

It is crucial that the programme can generate an accurate digital representation of the worm for further processing or analysis. A minor adjustment to the size of the structuring element can yield significant results to the digital representation. It is essential to experiment with different kernel sizes to find the optimal result across all the video sources.

For denoising, the effectiveness of the smallest 3x3 kernel, shown in Figure 4.6a above, is compared with a bigger kernel of 7x7 with the same shape.

a) b)

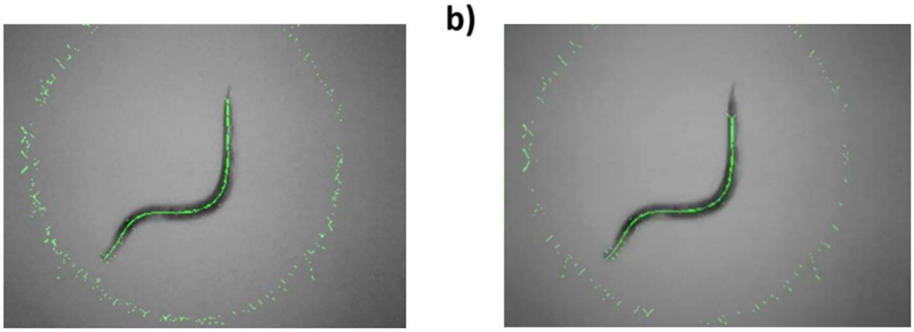

Figure 5.7a: *final result obtained from denoising using 3x3 kernel*

Figure 5.7b: *outcome obtained from denoising using 7x7 kernel*

For skeletonizing, the effectiveness of the smallest 3x3 kernel, shown in Figure 4.6b above, is compared with a bigger kernel of 5x5 with the same shape.

a) b)

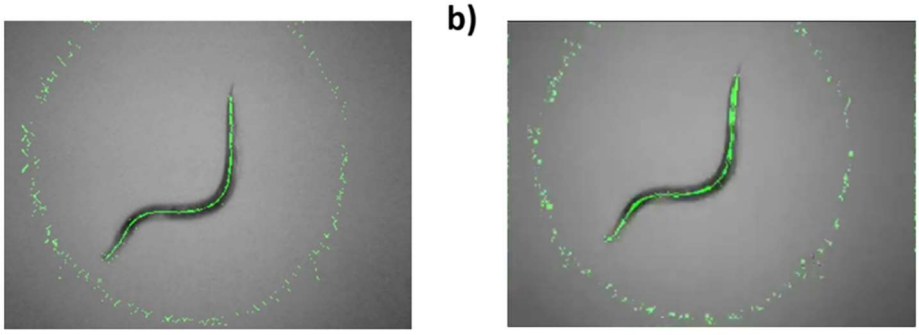

Figure 5.8a: *final result obtained from skeletonizing using 3x3 kernel*

Figure 5.8b: *outcome obtained from skeletonizing using 5x5 kernel*

It is evident that using bigger kernels would lead to loss of finer details and cause important feature distortions in the image. Therefore, the smaller kernels are used instead since preserving fine details of the skeleton is critical.

In Adjustment, basic colours such as blue, green and red are tested to determine which one provides the best clarity to the digital backbone representation when composited. The colour channels are represented as [255,0,0], [0,255,0] and [0,0,255] respectively. In Figure 5.9 below and Figure 5.4 above, it is evident that green is the optimal choice since it has the greatest contrast against the microscopic worm, commonly in dark-grey, regardless of the background colour in the original image.

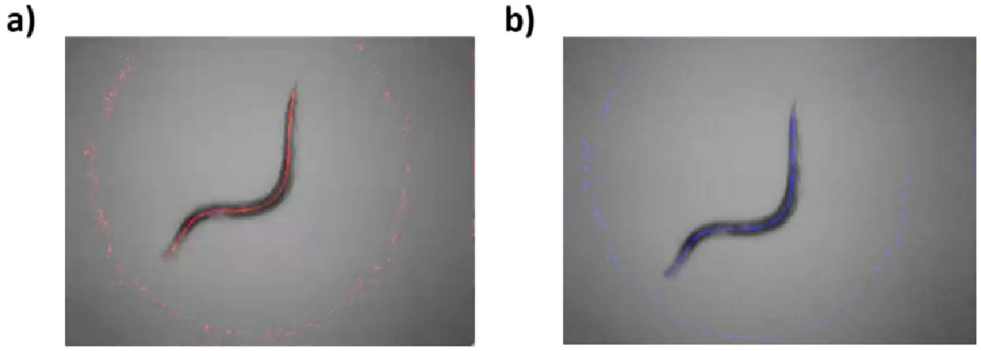

***Figure 5.9**: blue and red overlays*

2.5.2 Success and Failures

Additional samples are extracted from different video sources to test if the programme is working as intended. The programme consistently estimates the midline and creates a `skeleton' describing the worm's midline from head to tail correctly. Moreover, the programme successfully accomplishes this task even for worm postures that intersect or overlap themselves, as requested in the project brief by University of Leeds.

However, issues arise when external factors are involved. For instance, in Figure 5.11a on page 26, the programme cannot process the image content and detect the worm due to the black border at the bottom of the image. If the image is light enough in colour, the programme will process the image as background and mistakenly consider the border as the object. Another instance, in Figure 5.11b on page 26, the programme fails to detect the spine of the head and tail of the worm due to the shadows that are wrapping around the worm, resulting in a loss of digital representation. A possible

solution to resolve these issues is to design another supplementary programme to further preprocess the frame by cropping the image and removing any borders and shadows present around or within it. Since this task falls outside the scope of the project's aim, the supplementary programme will not be included as part of this project.

Figure 5.10: *successful examples*

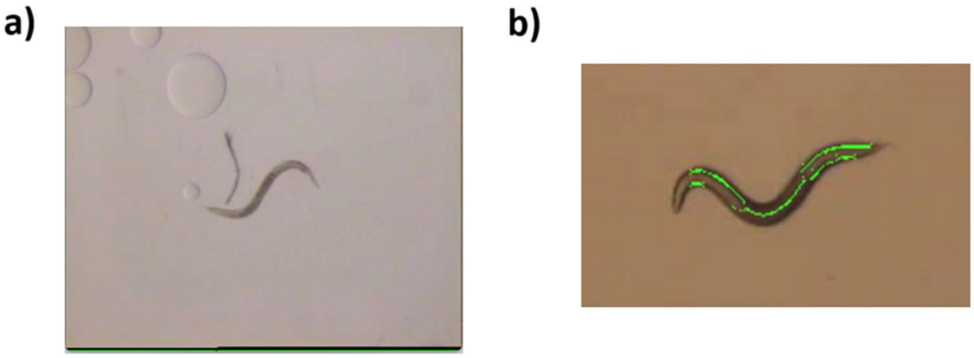

Figure 5.11a: *worm is unsuccessfully detected as the object*

Figure 5.11b: *spine of the worm is not fully detected*

2.6 Conclusion

2.6.1 About the Project

I am tasked with designing a supplementary program to support the research conducted at the University of Leeds on the locomotion of simulated worms and C. elegans. This research aims to provide insights into animal development and behaviour. The programme is responsible for generating a digital representation of the worm using skeletonization, a technique and tool prominent in various fields such as Biology and Computer Vision. By pursuing this project, I have learned the basics of developing a Computer Vision application with OpenCV, and problem-solve.

After carefully considering the project brief and system requirement, I have drawn inspiration from the popular Zhang-Suen thinning algorithm to implement the skeletonization programme, which consists of four phases. The first phase prepares the source by applying threshold and denoising techniques for subsequent phases. The second phase generates its digital representation based on the principle of the Hit-and-Miss transform. The colour of the representation is further adjusted in the third phase before composition for the final result in the fourth phase.

My programme has been quite successful according to the tests and results. It can clearly and accurately describe the midline from head to tail of the worm even in motion, despite some noise and external factors such as shadows and borders. Additionally, the programme is capable of detecting worms that are intersecting or overlapping, showcasing its advanced and robust capabilities.

2.6.2 The Current World

Skeletonization concepts have already been employed across various fields such as Civil engineering, Arts & Crafts and Artificial Intelligence. Below are some examples of new or recent innovative applications of skeletonization.

In Civil Engineering, skeletonization serves as a crucial tool for generating digital blueprints, capturing the essential structural elements of physical assets such as buildings, machinery, and infrastructure. This enables engineers to gain valuable insights for predictive maintenance and performance optimization.

In the field of Arts & Crafts, artists and designers can utilize skeletonization for creating abstract representations of real-world objects or environments. This approach empowers artists and designers to produce unique and innovative visual expressions and compositions while preserving the integrity or essence of the base model. Furthermore, skeletonization can optimize designs for 3D printing by reducing the complexity of objects while maintaining their structural integrity. This can result in faster printing times, lower material costs, and improved performance of printed parts in various applications, from aerospace to consumer products.

In the field of Artificial Intelligence, skeletonization techniques can be utilized to enhance AR and VR experiences by accurately overlaying virtual objects onto real-world scenes [15]. This can improve interaction and immersion in applications such as gaming, education, training simulations, and architectural visualization.

2.6.3 The Future World

This section aims to inspire and motivate our fellow readers and working professionals by exploring imaginative outcome possibilities through the application of skeletonization techniques.

Based on my understanding and inspiration drawn from this project, I strongly believe skeletonization has the potential to revolutionize the study of human DNA and make further contributions to future generations. Currently, skeletonization may not be directly applicable to the study of human DNA. However, advancements in technology and interdisciplinary research could potentially emerge at the intersection of computer vision and genomics. These developments could lead to innovative applications in genomics and DNA analysis and enhance our understanding of the human genome and its role in health and disease.

My grandmother suffered a deadly stroke caused by the rupture of blood vessels in her brain. Her condition has inspired me to explore potential applications of skeletonization for detecting early signs of vessel rupture.

By employing skeletonization on imaging scans, like MRI or CT scans, digital representation of the vascular structure can be generated. This representation can be used by the researchers to closely analyse the vessel's structure and pinpointing any abnormalities or weakened areas in the vessel

walls that may signal a high risk of rupture. If abnormality is identified, corrective actions can be taken accordingly to repair and reinforce the weakened vessel wall to reduce risk of rupture. These may include medication regimens, lifestyle modifications or surgical interventions, depending on the severity of the condition. I strongly believe that skeletonization have the potential to transform the medical field.

As technology continues to advance in various industries, skeletonization is anticipated to become increasingly widespread and even indispensable in the future.

3 References

[1] AIWizards.ai, “Revolutionizing Security: The Role of Computer Vision AI in Surveillance Systems,” LinkedIn, Dec. 14, 2023. https://www.linkedin.com/pulse/revolutionizing-security-role-computer-vision-ai-surveillance-j8hzc/ (accessed April 20, 2024).

[2] “Caenorhabditis elegans,” Wikipedia, Feb. 23, 2022. https://simple.wikipedia.org/wiki/Caenorhabditis_elegans (accessed April 22, 2024).

[3] Bio-Rad Laboratories, “OpenWorm: An open-source C. elegans nematode simulation,” YouTube. May 29, 2013. [Online]. Available: https://www.youtube.com/watch?v=Ejf6FwIibkE

[4] “Topological skeleton,” Wikipedia, Nov. 06, 2022. https://en.wikipedia.org/wiki/Topological_skeleton (accessed April 22, 2024).

[5] A. Lim, “Osteology: definition and applications,” ThoughtCo, Aug. 09, 2019. https://www.thoughtco.com/osteology-definition-and-applications-4588264 (accessed April 22, 2024).

[6] A. Bucksch, “A Practical Introduction to Skeletons for the Plant Sciences,” Applications in Plant Sciences, vol. 2, no. 8, p. 1400005, Aug. 2014, doi: 10.3732/apps.1400005.

[7] “3D Skeletonization /Volume Thinning,” Vizlab. https://vizlab.rutgers.edu/vol_thin (accessed April 23, 2024).

[8] Made Sudarma and Ni Putu Sutramiani, “The Thinning Zhang-Suen Application Method in the Image of Balinese Scripts on the Papyrus,” International journal of computer applications, vol. 91, no. 1, pp. 9–13, Apr. 2014, doi: 10.5120/15844-4726.

[9] R. Code, “Zhang-Suen thinning algorithm,” Rosetta Code, Feb. 17, 2024. https://rosettacode.org/wiki/Zhang-Suen_thinning_algorithm (accessed April 30, 2024).

[10] M. Gramblička and J. Vaský, “Comparison of thinning algorithms for vectorization of engineering drawings,” Journal of Theoretical and Applied Information Technology, vol. 31, no. 2, 2016, Accessed: April

30, 2024. [Online]. Available: https://www.jatit.org/volumes/Vol94No2/2Vol94No2.pdf

[11] "Otsu Thresholding," The Lab Book pages. http://www.labbookpages.co.uk/software/imgProc/otsuThreshold.html (accessed April 15, 2024).

[12] M. Krishnan, "Otsu's method for image thresholding explained and implemented," Muthukrishnan, Mar. 13, 2020. https://muthu.co/otsus-method-for-image-thresholding-explained-and-implemented/ (accessed April 15, 2024).

[13] "Hit-or-Miss," OpenCV. https://docs.opencv.org/4.x/db/d06/tutorial_hitOrMiss.html (accessed April 7, 2024).

[14] "Image Filtering," OpenCV. https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html (accessed April 7, 2024).

[15] Robert Costello, "Figure 2," ResearchGate. https://www.researchgate.net/figure/Skin-Bones-screen-capture-of-the-AR-experience-triggered-from-the-skeleton-of-a_fig2_329453256 (accessed May 9, 2024).